

SOFTWARE REQUIREMENTS

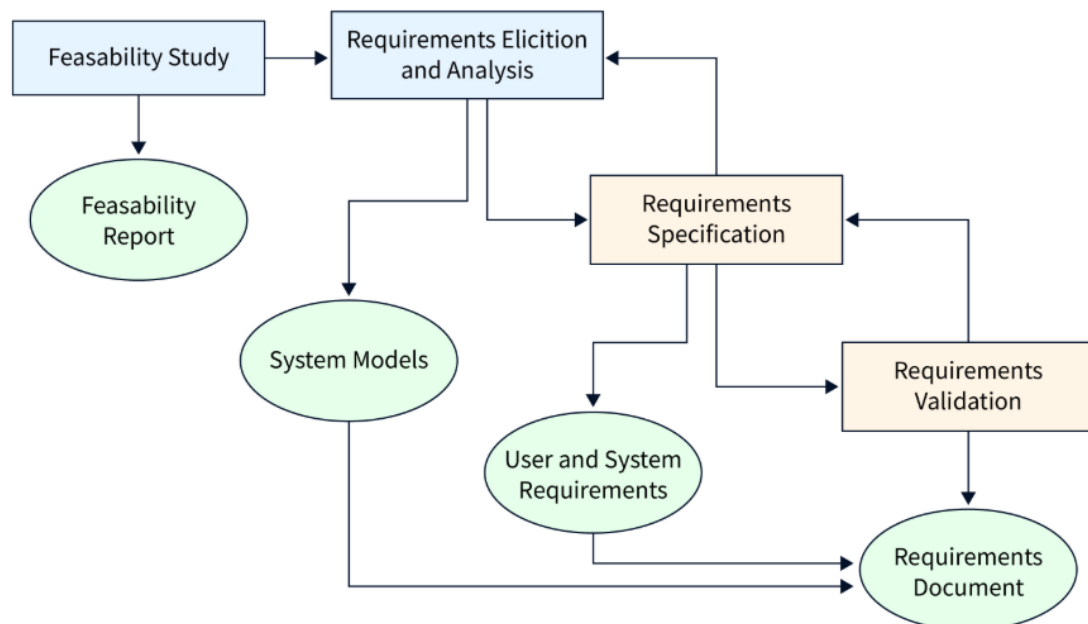
1. UNDERSTANDING REQUIREMENTS: REQUIREMENT ENGINEERING

Requirements engineering in software engineering is a critical and initial phase of project development, essential for successful systems. It serves as a bridge between stakeholders' aspirations and software possibilities, shaping project scope, schedule, and outcomes.

This process involves understanding user needs, defining system functionalities, and managing constraints to gather, document, analyze, and validate requisites systematically. Collaboration among stakeholders is crucial, and detailed documentation ensures that developers understand the project's essence.

Effective requirements engineering demands iterative refinement and validation, creating accurate and achievable project goals, while also preventing scope changes and project failures. It is an ongoing process throughout the software development lifecycle, where revisiting and refining requirements are key to the success of the software project.

Requirement Engineering Process



1.1 Requirement Engineering

Requirement Engineering is a fundamental process in software development that focuses on understanding, documenting, validating, and managing the needs and constraints of stakeholders to ensure the successful development of a software system. The process typically consists of several steps to ensure that the final software product meets the needs and expectations of the users and stakeholders.

A. Feasibility Study: This is the initial phase of the requirement engineering process. It involves evaluating the feasibility of the proposed software project. The main objective is to determine whether the project is technically, economically, and operationally viable. During this phase, the following aspects are considered:

Technical Feasibility: Can the proposed system be developed with the available technology and resources?

Economic Feasibility: Is the project economically viable in terms of cost and benefits?

Operational Feasibility: Will the system work effectively within the existing organizational structure and processes?

B. Requirement Elicitation and Analysis: This phase involves gathering and analyzing requirements from stakeholders, including users, customers, and domain experts. The goal is to understand the needs of the system and translate them into specific software requirements. This process involves several techniques, such as:

Interviews: Direct communication with stakeholders to gather information.

Surveys: Distributing questionnaires to collect opinions and preferences.

Workshops: Group sessions to brainstorm and gather requirements collaboratively.

Observation: Actively observing users to understand their workflow and needs.

Prototyping: Building a preliminary version of the software to elicit feedback.

Once requirements are gathered, they are analyzed for consistency, completeness, and feasibility. Conflicts and ambiguities are resolved, and the requirements are organized into different categories based on their characteristics (e.g., functional, non-functional).

C. Software Requirement Specification: In this phase, the gathered and analyzed requirements are documented formally. The Software Requirement Specification (SRS) document is created, which serves as a blueprint for the development process. The SRS includes the following components:

Functional Requirements: Descriptions of what the system should do in terms of specific functions and features.

Non-functional Requirements: Constraints and qualities the system must possess (e.g., performance, security).

User Interfaces: Descriptions of how users will interact with the system.

Data Requirements: Details about the data the system will store, process, and manage.

D. Software Requirement Validation: This step involves validating the documented requirements to ensure they accurately represent the stakeholders' needs and expectations. The aim is to identify and rectify any errors or misunderstandings before development begins. Techniques used for requirement validation include:

Reviews: Expert reviews and walkthroughs to identify issues in the SRS.

Prototyping: Building a working model of the software to validate requirements.

Simulation: Simulating the software's behavior to test its alignment with requirements.

E. Software Requirement Management: Requirement management is an ongoing process that involves maintaining and tracking changes to requirements throughout the software development lifecycle. It includes activities such as:

Version Control: Managing different versions of the requirements document.

Change Control: Managing and documenting changes to requirements and their impact.

Traceability: Establishing links between requirements and design, implementation, and testing.

Prioritization: Assigning priorities to requirements to guide development efforts.

Communication: Ensuring stakeholders are informed about changes and updates to requirements.

1.2 Challenges and Risks Involved in Requirement Engineering

a. Incomplete Requirements:

- Sometimes stakeholders may not fully articulate their needs or omit crucial details, leading to incomplete requirements.
- Incomplete requirements can result in systems that do not meet user expectations or fail to address critical functionalities.

b. Ambiguous Requirements:

- Vague or ambiguous language can lead to different interpretations of requirements by different team members.
- Ambiguities can result in miscommunication, design flaws, and delays in development.

c. Changing Requirements:

- Stakeholder needs and market conditions can evolve, causing requirements to change during the project lifecycle.
- Frequent changes can lead to scope creep, budget overruns, and development delays.

d. Conflicting Requirements:

- Different stakeholders may have conflicting needs or priorities that are challenging to reconcile.

- Resolving conflicts requires careful negotiation and trade-offs to ensure a balanced solution.

e. Poor Requirement Traceability:

- Ensuring that each requirement is appropriately tracked throughout the development process can be complex.
- Poor traceability can make it difficult to verify that all requirements have been addressed and tested, leading to quality issues.

1.3 Tools Involved in Requirement Engineering

Requirement Engineering (RE) is a crucial phase in software development that focuses on understanding, documenting, and managing the needs and expectations of stakeholders for a software system. To effectively perform requirement engineering, various tools are employed to facilitate the process and ensure that the project's goals are met. These tools aid in capturing, analysing, validating, and managing requirements throughout the software development lifecycle. Here, I'll detail some of the key tools involved in requirement engineering:

- **Document Editors and Word Processors:** Traditional document editors like Microsoft Word, Google Docs, and LaTeX are often used to create textual requirement documents. These tools help write and format textual descriptions of requirements, use cases, user stories, and other related documentation.
- **Spreadsheet Tools:** Tools like Microsoft Excel or Google Sheets can be used to create requirement lists, matrices, and traceability tables. These tools are useful for managing and organizing requirements, and they allow for easy sorting, filtering, and categorization.
- **Diagramming Tools:** Diagrams are powerful visual aids for understanding and representing requirements. Tools like Microsoft Visio, draw.io, and Lucidchart enable the creation of diagrams such as use case diagrams, data flow diagrams, activity diagrams, and entity-relationship diagrams to illustrate system behavior, interactions, and data flow.
- **Requirements Management Tools:** These tools are specifically designed to capture, organize, track changes, and manage requirements throughout the project lifecycle. Examples include IBM Engineering Requirements Management DOORS, Jama Connect, and Helix RM. These tools often support features like version control, traceability, and collaboration.
- **Prototyping Tools:** Prototyping tools like Balsamiq, Axure, and Sketch are useful for creating interactive mockups and prototypes of the user interface. Prototypes allow stakeholders to visualize how the software will look and behave, providing a tangible representation of requirements.
- **Modeling Languages:** Modeling languages like Unified Modeling Language (UML) provide a standardized way to represent system requirements and designs. UML diagrams such as use case diagrams, class diagrams, sequence diagrams, and state diagrams help in communicating complex requirements and system structures.
- **Collaboration Tools:** Collaboration tools like Confluence, SharePoint, and Microsoft Teams facilitate communication and collaboration among distributed teams. They allow team members to share and review requirement documents, provide feedback, and discuss changes.

- **Requirement Elicitation Tools:** Tools like surveys, interviews, questionnaires, and workshops are essential for gathering requirements directly from stakeholders. While these tools are not software applications, they are crucial techniques used to extract requirements information.
- **Validation and Verification Tools:** Tools such as formal methods, static analysis tools, and simulation tools help in validating and verifying requirements for correctness and consistency. These tools aid in identifying potential issues early in the process.
- **Traceability Tools:** Traceability tools help establish and manage the relationships between different levels of requirements, ensuring that all requirements are covered by design, implementation, and testing. These tools assist in maintaining a clear link between the various phases of software development.

1.4 Advantages of Requirements Engineering in Software Engineering

- **Customer Satisfaction:** Well-defined requirements lead to software that meets user needs, enhancing customer satisfaction.
- **Early Issue Detection:** Detecting issues in requirements early prevents costly errors and misunderstandings later in development.
- **Cost and Time Savings:** Detailed requirements reduce scope creep, saving time and resources by avoiding rework.
- **Effective Planning:** Clear requirements aid in accurate project planning, resource allocation, and effort estimation.
- **Traceability:** Requirements traceability links software components to specific requirements, aiding in impact analysis during changes.
- **Validation and Verification:** Accurate requirements serve as a basis for validating and verifying software correctness.
- **Collaboration:** Engaging stakeholders fosters collaboration, shared understanding, and effective teamwork.
- **Scope Management:** Formalized requirements prevent uncontrolled scope expansion, managing project scope effectively.
- **Risk Mitigation:** Thorough requirements facilitate risk identification and mitigation planning.
- **Quality Assurance:** Rigorous requirements lead to a higher-quality end product and informed testing efforts.
- **Regulatory Compliance:** Clear requirements assist in meeting industry regulations and standards.
- **Documentation and Maintenance:** Accurate requirements serve as valuable documentation for ongoing maintenance and enhancements.
- **User Involvement:** Engaging users ensures their needs are considered, resulting in a product aligned with expectations.
- **Change Management:** Requirements provide a structured approach to managing change requests.
- **Long-Term Benefits:** Investing in requirements engineering yields lasting benefits throughout the software's lifecycle.

1.5 Disadvantages of Requirements Engineering in Software Engineering

- **Ambiguity:** Poorly defined requirements can lead to confusion and misinterpretation among stakeholders and developers.
- **Scope Creep:** Uncontrolled additions to requirements can expand project scope, causing delays and increased costs.
- **Changing Requirements:** Shifting requirements during development can lead to delays, rework, and potential conflicts.
- **User Involvement Challenges:** Engaging users effectively can be difficult, leading to missed requirements and dissatisfaction.
- **Overemphasis on Documentation:** Excessive focus on documentation can slow down development and limit agility.
- **Miscommunication:** Inadequate communication can result in gaps between stakeholder expectations and developer understanding.
- **Time-Consuming Process:** Elaborate requirement gathering and analysis can prolong project initiation.
- **Resistance to Change:** Stakeholders might resist necessary changes due to emotional attachment to initial requirements.
- **Dependency on Tools:** Overreliance on requirement management tools can lead to issues if tools fail or become obsolete.
- **Neglecting Non-Functional Requirements:** Focusing solely on functional aspects might lead to overlooking critical non-functional aspects like performance and security.

1.6 Compare Different Requirement Engineering Approaches

a. Waterfall Model:

- Sequential and linear approach.
- Emphasizes complete documentation before moving to the next phase.
- Changes are difficult to incorporate once a phase is completed.

b. Agile Methodology:

- Iterative and incremental approach.
- Prioritizes working software over comprehensive documentation.
- Flexibility to adapt to changing requirements throughout the project.

c. Prototyping:

- Focuses on building a preliminary version to gather feedback.
- Useful for exploring unclear or evolving requirements.
- May lead to scope creep if not managed well.

d. Spiral Model:

- Combines iterative development with risk management.

- Iterations are split into phases, each addressing risks.
- Suited for projects with high uncertainty and evolving requirements.

e. Use Case-driven:

- Centers around identifying interactions between users and the system.
- Expresses requirements as use cases and scenarios.
- Helps in capturing functional aspects and user behavior.

1.7 Prerequisite of Software Requirements

The collection of software requirements is the cornerstone of successful software development. To establish this foundation, certain prerequisites must be met:

- **Clarity:** Requirements should be unambiguous to avoid confusion.
- **Correctness:** Accuracy is essential to ensure the software meets intended needs.
- **Consistency:** Internal harmony prevents contradictions within requirements.
- **Coherence:** Requirements should logically align with project goals.
- **Comprehensibility:** All stakeholders should easily understand the requirements.
- **Modifiability:** Requirements must be adaptable without disrupting the process.
- **Verifiability:** Objective methods should verify if requirements are met.
- **Prioritization:** Focus on critical features even with resource limitations.
- **Unambiguity:** Avoid multiple interpretations to ensure clarity.
- **Traceability:** Connect requirements to development stages for accountability.
- **Credible Source:** Source of requirements should be reliable and authoritative.

1.8 Software requirements are categorized into:

Functional Requirements: Define software functions and behaviors.

Non-functional Requirements: Cover aspects like security, usability, and scalability. Non-functional requirements have two categories:

- **Execution Qualities:** Observable at runtime, like security.
- **Evolution Qualities:** Impact software's long-term viability and maintenance.

Meeting these prerequisites and understanding requirement categories lays the foundation for a successful software project, delivering a solution that aligns with stakeholder needs and expectations.

2. ESTABLISHING GROUND WORK

Establishing groundwork in software engineering involves identifying stakeholders, understanding client needs through requirements gathering, defining the project scope by answering key questions about the problem and solution, and choosing an appropriate development methodology and tools. The steps required to establish the groundwork for an understanding of software requirements.

a. Identifying Stakeholders

- Business operations managers, product managers, marketing people, internal and external customers, end users, consultants, product engineers, software engineers, support and maintenance engineers, and others.
- Each stakeholder has a different view of the system, achieves different benefits when the system is successfully developed, and is open to different risks if the development effort should fail.

b. Recognizing Multiple Viewpoints

- Because many different stakeholders exist, the requirements of the system will be explored from many different points of view.

For example, the marketing group is interested in functions and features that will excite the potential market, making the new system easy to sell and End users may want features that are easy to learn and use.

- As information from multiple viewpoints is collected, requirements may be inconsistent or may conflict with one another.
- categorize all stakeholder information , that will allow decision makers to choose an internally consistent set of requirements for the system.

c. Working toward Collaboration

- The job of a requirements engineer is to identify areas of commonality (i.e., requirements on which all stakeholders agree) and areas of conflict or inconsistency (i.e., requirements that are desired by one stakeholder but conflict with the needs of another stakeholder).
- stakeholders collaborate by providing their view of requirements, but a strong “project champion”(e.g., a business manager or a senior technologist) may make the final decision about which requirements make the cut.

3. REQUIREMENTS ELICITATION

Requirements elicitation is the process of gathering and defining the requirements for a software system. The goal of requirements elicitation is to ensure that the software development process is based on a clear and comprehensive understanding of the customer's needs and requirements.

What is Requirement Elicitation?

The process of investigating and learning about a system's requirements from users, clients, and other stakeholders is known as requirements elicitation. Requirements elicitation in software engineering is perhaps the most difficult, most error-prone, and most communication-intensive software development.

- Requirement Elicitation can be successful only through an effective customer-developer partnership. It is needed to know what the users require.
- Requirements elicitation involves the identification, collection, analysis, and refinement of the requirements for a software system.
- Requirement Elicitation is a critical part of the software development life cycle and is typically performed at the beginning of the project.
- Requirements elicitation involves stakeholders from different areas of the organization, including business owners, end-users, and technical experts.

3.1 Importance of Requirements Elicitation

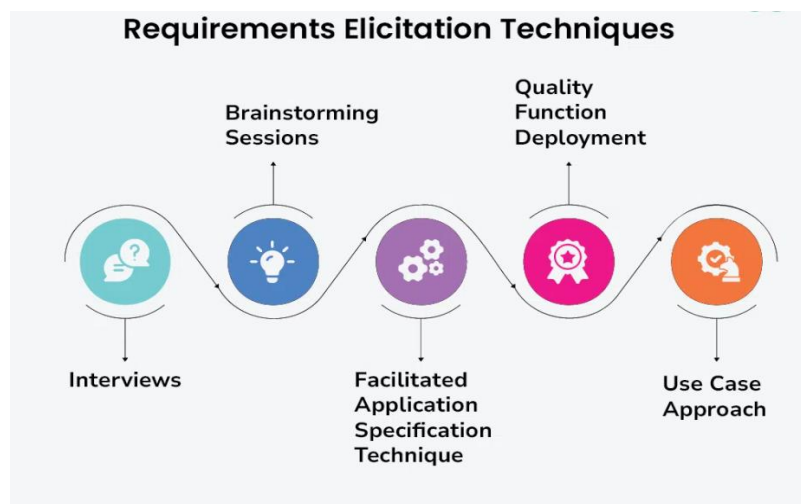
- **Compliance with Business Objectives:** The process of elicitation guarantees that the software development endeavors are in harmony with the wider company aims and objectives. Comprehending the business context facilitates the development of a solution that adds value for the company.
- **User Satisfaction:** It is easier to create software that fulfills end users' needs and expectations when they are involved in the requirements elicitation process. Higher user pleasure and acceptance of the finished product are the results of this.
- **Time and Money Savings:** Having precise and well-defined specifications aids in preventing miscommunication and rework during the development phase. As a result, there will be cost savings and the project will be completed on time.
- **Compliance and Regulation Requirements:** Requirements elicitation is crucial for projects in regulated industries to guarantee that the software conforms with applicable laws and norms. In industries like healthcare, finance, and aerospace, this is crucial.
- **Traceability and Documentation:** Throughout the software development process, traceability is based on well-documented requirements. Traceability helps with testing, validation, and maintenance by ensuring that every part of the software can be linked to a particular requirement.

3.2 Requirements Elicitation Activities

- Requirements elicitation includes the subsequent activities. A few of them are listed below:
- Knowledge of the overall area where the systems are applied.
- The details of the precise customer problem where the system is going to be applied must be understood.
- Interaction of system with external requirements.
- Detailed investigation of user needs.
- Define the constraints for system development.

3.3 Requirements Elicitation Methods

There are several requirements elicitation methods. A few of them are listed below:



1. Interviews

The objective of conducting an interview is to understand the customer's expectations of the software.

- It is impossible to interview every stakeholder hence representatives from groups are selected based on their expertise and credibility. Interviews may be open-ended or structured.
- In open-ended interviews, there is no pre-set agenda. Context-free questions may be asked to understand the problem.
- In a structured interview, an agenda of fairly open questions is prepared. Sometimes a proper questionnaire is designed for the interview.

2. Brainstorming Sessions

Brainstorming Sessions is a group technique

- It is intended to generate lots of new ideas hence providing a platform to share views
- A highly trained facilitator is required to handle group bias and conflicts.
- Every idea is documented so that everyone can see it.
- Finally, a document is prepared which consists of the list of requirements and their priority if possible.

3. Facilitated Application Specification Technique

Its objective is to bridge the expectation gap - the difference between what the developers think they are supposed to build and what customers think they are going to get. A team-oriented approach is developed for requirements gathering. Each attendee is asked to make a list of objects that are:

- Part of the environment that surrounds the system.
- Produced by the system.
- Used by the system.

Each participant prepares his/her list, different lists are then combined, redundant entries are eliminated, the team is divided into smaller sub-teams to develop mini-specifications and finally, a draft of specifications is written down using all the inputs from the meeting.

4. Quality Function Deployment

In this technique customer satisfaction is of prime concern, hence it emphasizes the requirements that are valuable to the customer.

3 types of requirements are identified:

Normal requirements: In this the objective and goals of the proposed software are discussed with the customer. For example - normal requirements for a result management system may be entry of marks, calculation of results, etc.

Expected requirements: These requirements are so obvious that the customer need not explicitly state them. Example - protection from unauthorized access.

Exciting requirements: It includes features that are beyond customer's expectations and prove to be very satisfying when present. For example - when unauthorized access is detected, it should back up and shut down all processes.

5. Use Case Approach

Use Case technique combines text and pictures to provide a better understanding of the requirements. The use cases describe the 'what', of a system and not 'how'. Hence, they only give a functional view of the system. The components of the use case design include three major things - Actor, use cases, and use case diagram.

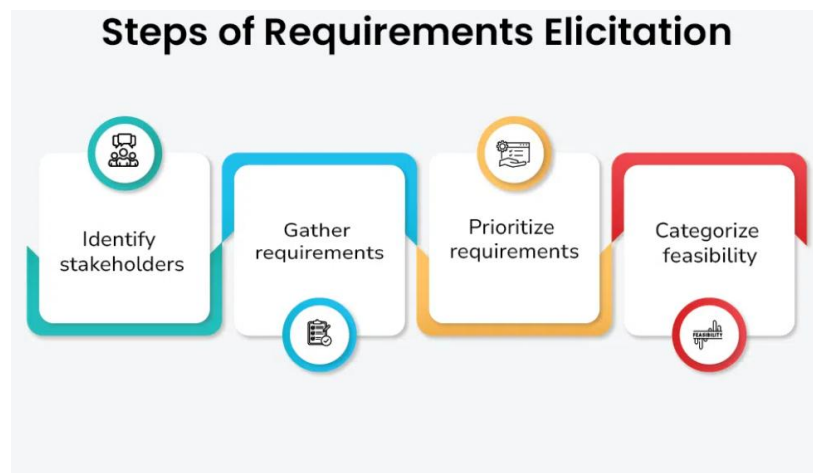
Actor: It is the external agent that lies outside the system but interacts with it in some way. An actor may be a person, machine, etc. It is represented as a stick figure. Actors can be primary actors or secondary actors.

- **Primary actors:** It requires assistance from the system to achieve a goal.
- **Secondary actor:** It is an actor from which the system needs assistance.

Use cases: They describe the sequence of interactions between actors and the system. They capture who(actors) do what(interaction) with the system. A complete set of use cases specifies all possible ways to use the system.

Use case diagram: A use case diagram graphically represents what happens when an actor interacts with a system. It captures the functional aspect of the system.

3.4 Steps of requirements elicitation



1. Identify Stakeholders

The first step is to figure out who all the key players are in the project. These are the people who will either use the product, help develop it, or be impacted by it. This could include users, developers, project managers, and customers. It's important to get everyone involved early on, so you gather all the necessary input and perspectives when defining the requirements.

2. Gather Requirements

Once you've identified the stakeholders, the next step is to gather their requirements. This means understanding what they need from the system or product. Requirements can be functional (what the system should do) or non-functional (how it should perform). You can use different methods like interviews, surveys, or group discussions to collect all these needs.

3. Prioritize Requirements

Not all requirements are equal in terms of importance. After gathering them, you'll need to prioritize which ones should be addressed first. This helps ensure that the most important features are built into the system first, especially when resources (like time and budget) are limited.

4. Categorize Feasibility

After prioritizing, the next step is to assess how realistic each requirement is. You'll categorize them into three groups:

Achievable: These are requirements that are realistic and can be done within the available resources (time, budget, etc.).

Deferred: These requirements are important but can't be implemented in this phase. They'll be put on hold for now with a clear reason why.

Impossible: These requirements can't be done because of technical or other limitations, and they should be removed from the list.

3.5 Advantages of Requirements Elicitation

- **Clear requirements:** Helps to clarify and refine customer requirements.
- **Improves communication:** Improves communication and collaboration between stakeholders.
- **Results in good quality software:** Increases the chances of developing a software system that meets customer needs.
- **Avoids misunderstandings:** Avoids misunderstandings and helps to manage expectations.
- **Supports the identification of potential risks:** Supports the identification of potential risks and problems early in the development cycle.
- **Facilitates development of accurate plan:** Facilitates the development of a comprehensive and accurate project plan.
- **Increases user confidence:** Increases user and stakeholder confidence in the software development process.
- **Supports identification of new business opportunities:** Supports the identification of new business opportunities and revenue streams.

3.6 Disadvantages of Requirements Elicitation

- **Time-consuming:** It can be time-consuming and expensive.
- **Skills required:** Requires specialized skills and expertise.
- **Impacted by changing requirements:** This may be impacted by changing business needs and requirements.

- **Impacted by other factors:** Can be impacted by political and organizational factors.
- **Lack of commitment from stakeholders:** This can result in a lack of buy-in and commitment from stakeholders.
- **Impacted by conflicting priorities:** Can be impacted by conflicting priorities and competing interests.
- **Sometimes inaccurate requirements:** This may result in incomplete or inaccurate requirements if not properly managed.
- **Increased development cost:** This can lead to increased development costs and decreased efficiency if requirements are not well-defined.

4. DEVELOPING USECASES

- “A use case describes the system’s behavior under various conditions as the system responds to a request from one of its stakeholders.”
 - A use case tells a stylized story about how an end user interacts with the system under a specific set of situations.
 - The first step in writing a use case is to define the set of “actors” that will be involved in the story.
 - Actors are the different people represent the roles that people (or devices) play as the system operates.
 - Every actor has one or more goals when using the system.
 - Once actors have been identified, use cases can be developed.
 - number of questions that should be answered by a use case:
- Who is the primary actor, the secondary actor(s)?
 - What are the actor’s goals?
 - What preconditions should exist before the story begins?
 - What main tasks or functions are performed by the actor?

Basic Safe Home requirements, we define four actors: **homeowner** (a user), **setup manager** (likely the same person as **homeowner**, but playing a different role), **sensors** (devices attached to the system), and the **monitoring and response subsystem** (the central station that monitors the SafeHome home security function). For the purposes of this example, we consider only the **homeowner actor**. The **homeowner** actor interacts with the home security function in a number of different ways using either the alarm control panel or a PC:

Considering the situation in which the homeowner uses the control panel, the basic use case for system activation follows:

1. The homeowner observes the SafeHome control panel to determine if the system is ready for input. If the system is not ready, a not ready message is displayed on the LCD display.
2. The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect and reset itself for additional input. If the password is correct, the control panel awaits further action.

3. The homeowner select the keys in stay or away to activate the system.

- Stay activates only perimeter sensors (inside motion detecting sensors are deactivated).
- Away activates all sensors.

4. When activation occurs, a red alarm light can be observed by the homeowner. The basic use case presents a high-level story that describes the interaction between the actor and the system.

5. BUILDING THE REQUIREMENT MODEL

- The analysis model is to provide a description of the required information, functional, and behavioral domains for a computer-based system.
- The model changes dynamically as learn more about the system to be built, and other stakeholders understand what they really require.

5.1 Elements of the Requirements Model

- There are many different ways to look at the requirements for a computer based system.
- Different modes of representation force to consider requirements from different viewpoints and to find inconsistencies and ambiguity of the gathered requirements.
- **Scenario-based elements:** The system is described from the user's point of view using a scenario-based approach. For example, basic use case.
- **Class-based elements:** Each usage scenario implies a set of objects that are manipulated as an actor interacts with the system. These objects are categorized into classes—a collection of things that have similar attributes and common behaviors.
- **Behavioral elements:** The behavior of a computer-based system can have a effect on the design that is chosen and the implementation approach that is applied. Therefore, the requirements model must provide modelling elements that represent behavior.

5.2 Analysis Patterns

Anyone who has done requirements engineering on more than few software projects then identify the some problems reoccur across all projects within a application domain.

- These analysis patterns suggest solutions (e.g., a class, a function, a behavior) within the application domain that can be reused when modelling many applications.
- Analysis patterns are integrated into the analysis model by reference to the pattern name. They are also stored in a repository so that requirements engineers can use.

Advantages of Analysis patterns are:

- Analysis patterns speed up the development of abstract analysis models that capture the main requirements of the concrete problem
- Analysis patterns facilitate the transformation of the analysis model into a design model by suggesting design patterns and reliable solutions for common problems.

6. NEGOTIATING REQUIREMENTS

- The negotiation is to develop a project plan that meets stakeholder needs while at the same time reflecting the real-world constraints (e.g., time, people, budget) that have been placed on the software team.
- The best negotiations strive for a “win-win” result.
- That is, stakeholders win by getting the system or product that satisfies the majority of their needs and you (as a member of the software team) win by working to realistic and achievable budgets and deadlines.
- Boehm defines a set of negotiation activities at the beginning of each software process iteration.

The following activities are defined:

1. Identification of the system or subsystem’s key stakeholders.
2. Determination of the stakeholders’ “win conditions.”
3. Negotiation of the stakeholders’ win conditions to reconcile them into a set of win-win conditions for all concerned (including the software team). Successful completion of these initial steps achieves a win-win result.

7. REQUIREMENTS VALIDATION

What is Requirements Validation?

Requirements validation is the process of checking that requirements defined for development, define the system that the customer wants. To check issues related to requirements, we perform requirements validation. We typically use requirements validation to check errors at the initial phase of development as the error may increase excessive rework when detected later in the development process. In the requirements validation process, we perform a different type of test to check the requirements mentioned in the Software Requirements Specification (SRS), these checks include:

- Completeness checks
- Consistency checks
- Validity checks
- Realism checks
- Ambiguity checks
- Variability

The output of requirements validation is the list of problems and agreed-on actions of detected problems. The lists of problems indicate the problem detected during the process of requirement validation. The list of agreed actions states the corrective action that should be taken to fix the detected problem.

7.1 Requirement Validation Techniques

There are several techniques that are used either individually or in conjunction with other techniques to check entire or part of the system:

- Test Case Generation
- Prototyping
- Requirements Reviews
- Automated Consistency Analysis
- Walk-through
- Simulation
- Checklists for Validation

1. Test Case Generation

The requirement mentioned in the SRS document should be testable, the conducted tests reveal the error present in the requirement. It is generally believed that if the test is difficult or impossible to design, this usually means that the requirement will be difficult to implement and it should be reconsidered.

2. Prototyping

In this validation technique the prototype of the system is presented before the end-user or customer, they experiment with the presented model and check if it meets their need. This type of model is mostly used to collect feedback about the requirement of the user.

3. Requirements Reviews

In this approach, the SRS is carefully reviewed by a group of people including people from both the contractor organizations and the client side, the reviewer systematically analyses the document to check errors and ambiguity.

4. Automated Consistency Analysis

This approach is used for the automatic detection of an error, such as non-determinism, missing cases, a type error, and circular definitions, in requirements specifications. First, the requirement is structured in formal notation then the CASE tool is used to check the in-consistency of the system, The report of all inconsistencies is identified, and corrective actions are taken.

5. Walk-through

A walkthrough does not have a formally defined procedure and does not require a differentiated role assignment.

- Checking early whether the idea is feasible or not.
- Obtaining the opinions and suggestions of other people.
- Checking the approval of others and reaching an agreement.

6. Simulation

Simulating system behavior in order to verify requirements is known as simulation. This method works especially well for complicated systems when it is possible to replicate real-world settings and make sure the criteria fulfil the desired goals.

7. Checklists for Validation

It employs pre-made checklists to methodically confirm that every prerequisite satisfies predetermined standards. Aspects like completeness, clarity and viability can all be covered by checklists.

7.2 Importance of Requirements Validation Techniques

- **Accuracy and Clarity:** It makes sure that the requirements are precise, unambiguous and clear. This helps to avoid miscommunications and misunderstandings that may result in mistakes and more effort in subsequent phases of the project.
- **User Satisfaction:** It confirms that the requirements meet the wants and expectations of the users, which helps to increase user happiness. This aids in providing a product that satisfies consumer needs and improves user experience as a whole.
- **Early Issue Identification:** It makes it easier to find problems, ambiguities or conflicts in the requirements early on. It is more economical to address these issues early in the development phase rather than later, when the project is far along.
- **Prevents the Scope Creep:** It ensures that the established requirements are well stated and recorded, which helps to prevent scope creep. By establishing defined parameters for the project's scope, requirements validation helps to lower the possibility of uncontrollably changing course.
- **Improving Quality:** It enhances the software product's overall quality. By detecting and resolving possible quality problems early in the development life cycle, requirements validation contributes to the creation of a more durable and dependable final product.

8. REQUIREMENTS MODELING

8.1 Requirements analysis

Requirements analysis results in the specification of software's operational characteristics, indicates software's interface with other system elements, and establishes constraints that software must meet. Requirements analysis allows you to elaborate on basic requirements established during the inception, elicitation, and negotiation tasks that are part of requirements engineering.

The requirements modeling action results in one or more of the following types of models:

- **Scenario-based models** of requirements from the point of view of various system "actors"
- **Data models** that depict the information domain for the problem
- **Class-oriented models** that represent object-oriented classes (attributes and operations) and the manner in which classes collaborate to achieve system requirements
- **Flow-oriented models** that represent the functional elements of the system and how they transform data as it moves through the system
- **Behavioral models** that depict how the software behaves as a consequence of external "events"

These models provide a software designer with information that can be translated to architectural, interface, and component-level designs. Finally, the requirements model provides the developer and the customer with the means to assess quality once software is built.

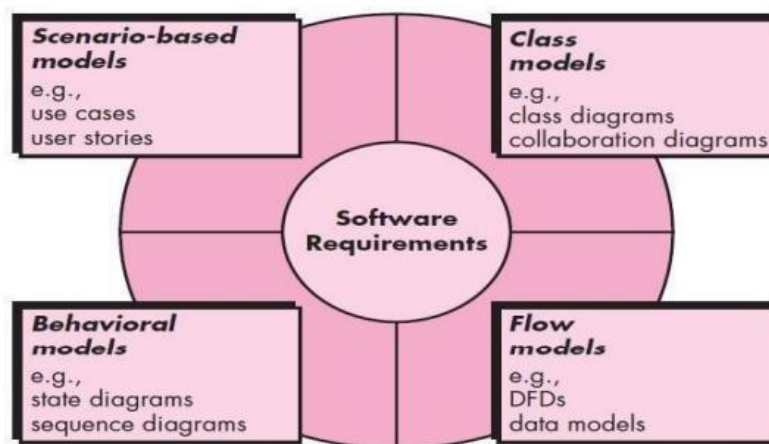
Throughout requirements modeling, primary focus is on what, not how. What user interaction occurs in a particular circumstance, what objects does the system manipulate, what functions must the system perform, what behaviors does the system exhibit, what interfaces are defined, and what constraints apply?

8.2 Requirements Modeling Approaches

One view of requirements modeling, called **structured analysis**, considers data and the processes that transform the data as separate entities. Data objects are modeled in a way that defines their attributes and relationships.

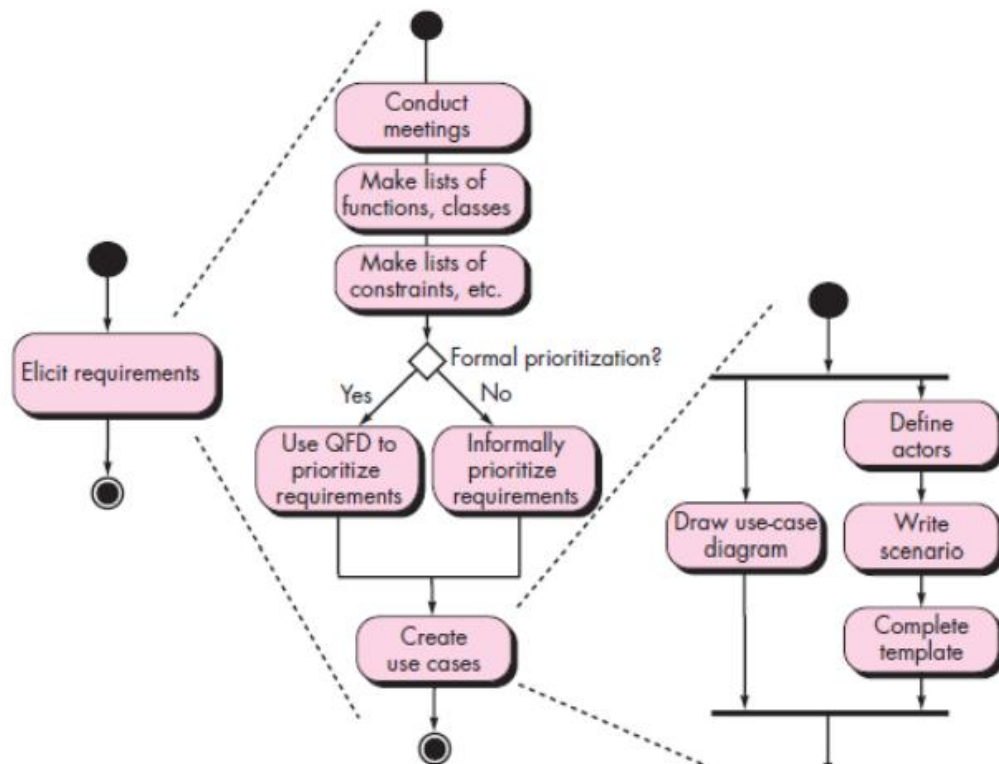
A second approach to analysis modeling, called object-oriented analysis, focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements. UML and the Unified Process are predominantly object oriented. Each element of the requirements model is represented in following figure presents the problem from a different point of view.

- **Scenario-based elements** depict how the user interacts with the system and the specific sequence of activities that occur as the software is used.
- **Class-based elements** model the objects that the system will manipulate, the operations that will be applied to the objects to effect the manipulation, relationships between the objects, and the collaborations that occur between the classes that are defined.
- **Behavioral elements** depict how external events change the state of the system or the classes that reside within it. Finally,
- **Flow-oriented elements** represent the system as an information transform, depicting how data objects are transformed as they flow through various system functions.



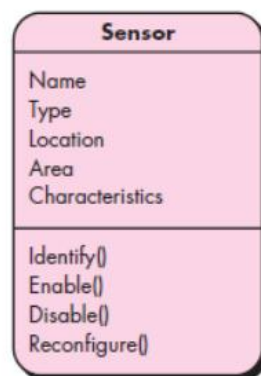
9. SCENARIO-BASED MODELING

Scenario-based elements: The system is described from the user's point of view using a scenario-based approach. For example, basic use cases and their corresponding use-case diagrams evolve into more elaborate template-based use cases. Scenario-based elements of the requirements model are often the first part of the model that is developed. Three levels of elaboration are shown, culminating in a scenario-based representation.



10. CLASS-BASED ELEMENTS

Class-based elements. Each usage scenario implies a set of objects that are manipulated as an actor interacts with the system. These objects are categorized into classes—a collection of things that have similar attributes and common behaviors.

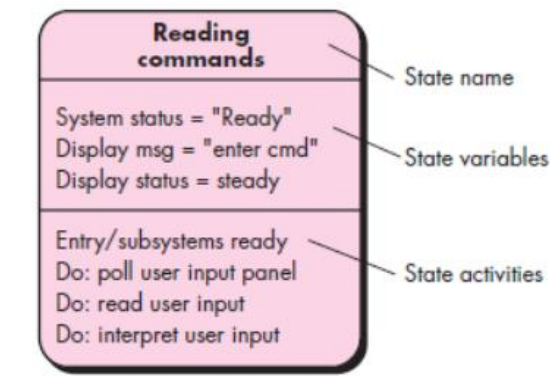


Class diagram for sensor

11. BEHAVIORAL ELEMENTS

Behavioral elements. The behavior of a computer-based system can have a profound effect on the design that is chosen and the implementation approach that is applied. Therefore, the requirements model must provide modeling elements that depict behavior. The state diagram is one method for representing the behavior of a system by depicting its states and the events that cause the system to

change state. A state is any externally observable mode of behavior. In addition, the state diagram indicates actions taken as a consequence of a particular event.



UML State diagram notation

12. FLOW-ORIENTED ELEMENTS

Flow-oriented elements. Information is transformed as it flows through a computer-based system. The system accepts input in a variety of forms, applies functions to transform it, and produces output in a variety of forms. Input may be a control signal transmitted by a transducer, a series of numbers typed by a human operator, a packet of information transmitted on a network link, or a voluminous data file retrieved from secondary storage. The transform(s) may comprise a single logical comparison, a complex numerical algorithm, or a rule-inference approach of an expert system.